

- [20] Wuyao Mai, Geng Hong, Qi Liu, Jinsong Chen, Jiarun Dai, Xudong Pan, Yuan Zhang, and Min Yang. Shell or nothing: Real-world benchmarks and memory-activated agents for automated penetration testing, 2025. URL: <https://arxiv.org/abs/2509.09207>, arXiv:2509.09207.
- [21] Iman Mirzadeh, Keivan Alizadeh, Hooman Shahrokhi, Oncel Tuzel, Samy Bengio, and Mehrdad Farajtabar. Gsm-symbolic: Understanding the limitations of mathematical reasoning in large language models, 2025. URL: <https://arxiv.org/abs/2410.05229>, arXiv:2410.05229.
- [22] Lajos Muzsai, David Imolai, and András Lukács. Hacksynth: Llm agent and evaluation framework for autonomous penetration testing. *arXiv preprint arXiv:2412.01778*, 2024.
- [23] Sho Nakatani. RapidPen: Fully automated IP-to-shell penetration testing with LLM-based agents. *arXiv preprint arXiv:2502.16730*, 2025.
- [24] Sho Nakatani. Rapidpen: Fully automated ip-to-shell penetration testing with llm-based agents. *arXiv preprint arXiv:2502.16730*, 2025.
- [25] Orange Cyberdefense. GOAD - game of active directory. <https://github.com/Orange-Cyberdefense/GOAD>, 2024. A pentest Active Directory LAB project providing vulnerable AD environments for practicing attack techniques.
- [26] OWASP Foundation. OWASP web security testing guide. <https://owasp.org/www-project-web-security-testing-guide/>, 2021. Version 4.2. Comprehensive guide to testing the security of web applications and web services.
- [27] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems, 2024. URL: <https://arxiv.org/abs/2310.08560>, arXiv:2310.08560.
- [28] PTES Technical Guideline Development Team. Penetration testing execution standard (PTES). <http://www.pentest-standard.org>, 2012. A comprehensive standard for conducting penetration tests, defining seven main phases from pre-engagement to reporting.
- [29] Minghao Shao, Boyuan Chen, Sofija Jancheska, Brendan Dolan-Gavitt, Siddharth Garg, Ramesh Karri, and Muhammad Shafique. An empirical evaluation of LLMs for solving offensive security challenges. *arXiv preprint arXiv:2402.11814*, 2024.
- [30] Xiangmin Shen, Lingzhi Wang, Zhenyuan Li, Yan Chen, Wencheng Zhao, Dawei Sun, Jiashui Wang, and Wei Ruan. PentestAgent: Incorporating LLM agents to automated penetration testing. In *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security (ASIA CCS '25)*, pages 375–391. ACM, 2025.
- [31] Georg Wölflein, Dyke Ferber, Daniel Truhn, Ognjen Arandjelovic, and Jakob Nikolas Kather. LLM agents making agent tools. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 26092–26130, Vienna, Austria, July 2025. Association for Computational Linguistics. URL: <https://aclanthology.org/2025.acl-long.1266/>, doi: 10.18653/v1/2025.acl-long.1266.
- [32] Benlong Wu, Guoqiang Chen, Kejiang Chen, Xiuwei Shang, Jiapeng Han, Yanru He, Weiming Zhang, and Nenghai Yu. AutoPT: How far are we from the end2end automated web penetration testing? *arXiv preprint arXiv:2411.01236*, 2024.
- [33] Qiusi Zhan, Richard Fang, Henil Shalin Panchal, and Daniel Kang. Adaptive attacks break defenses against indirect prompt injection attacks on llm agents, 2025. URL: <https://arxiv.org/abs/2503.00061>, arXiv:2503.00061.
- [34] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. Webarena: A realistic web environment for building autonomous agents. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

A Surveyed LLM-Based Penetration Testing Systems

Table 10 presents the complete list of 28 candidate systems identified in our survey. Systems meeting our inclusion criteria (LLM as core component, targeting penetration testing or CTF challenges, with published technical details) are marked with ✓.

B Tool and Skill Layer: Supported Tools

Table 11 lists the 38 security tools integrated into PENTEST-GPT v2’s Tool and Skill Layer. Each tool is exposed through a typed interface specifying input parameters, output schema, and pre/postconditions. Tool selection reflects standard penetration testing methodology and aligns with tools commonly used in professional certifications (e.g., OSCP) and real-world assessments.

Table 10: Complete list of surveyed LLM-based penetration testing systems. Systems marked with ✓ meet our inclusion criteria and are analyzed in Section 3.

System	Source	Year	Included
PentestGPT [8]	USENIX Security	2024	✓
AutoPT [32]	arXiv	2024	✓
RapidPen [24]	arXiv	2025	✓
PentestAgent [30]	arXiv	2024	✓
VulnBot [17]	arXiv	2025	✓
xOffense [19]	arXiv	2025	✓
TermiAgent [20]	arXiv	2025	✓
HackSynth [22]	arXiv	2024	✓
MAPTA [7]	arXiv	2025	✓
Cochise [12]	arXiv	2025	✓
<i>Excluded: Vulnerability detection only</i>			
VulnScanner-AI	GitHub	2024	
LLM-SecAudit	arXiv	2024	
CodeVuln	arXiv	2024	
BugHunter	RAID	2024	
AutoFuzz-LLM	CCS	2024	
<i>Excluded: Commercial/no details</i>			
Pentera	Commercial	2024	
Cobalt Strike AI	Commercial	2024	
CrowdStrike Charlotte	Commercial	2024	
<i>Excluded: Non-exploitation focus</i>			
CTF-Helper	arXiv	2023	
CryptoSolver	arXiv	2024	
RevEngGPT	arXiv	2024	
MalwareGPT	arXiv	2024	
ThreatGPT	arXiv	2024	
SecurityBot	GitHub	2024	
DFIR-Assistant	arXiv	2024	
IRBot	arXiv	2025	
SOC-Copilot	arXiv	2024	
VulnReport-LLM	arXiv	2024	

C Evidence Confidence Scoring

Table 12 presents the complete evidence confidence scoring rubric used by the TDA mechanism. Scores are assigned deterministically based on evidence type, enabling reproducible difficulty assessment.

Path Confidence Computation. For a path $P = (n_0, n_1, \dots, n_k)$ from root to current node, the evidence confidence is computed as:

$$E(P) = \frac{1}{k} \sum_{i=1}^k e(n_i) \quad (3)$$

where $e(n_i)$ is the confidence score assigned to node n_i based on Table 12. The root node n_0 is excluded as it represents the initial state before any evidence is gathered.

Tool Output Parsing. Evidence types are determined automatically by parsing tool outputs against expected patterns.

For example, `nmap` output containing “open” with a service version triggers version-matched vulnerability lookup (0.5); `sqlmap` output containing “injectable” triggers confirmed injection (0.8); successful `ssh` connection triggers valid credentials (1.0). The Tool Layer’s typed interfaces (Section 4.2) provide structured outputs that simplify this parsing.

Example. Consider a path: *port scan* → *web server (nginx 1.18)* → *directory bruteforce* → *login form discovered* → *SQL injection confirmed*. Evidence scores are: 0.3 (service identified), 0.5 (version-matched to known nginx vulnerabilities), 0.3 (endpoint exists), 0.8 (injection confirmed). Path confidence $E = (0.3 + 0.5 + 0.3 + 0.8)/4 = 0.475$, indicating moderate confidence appropriate for transitioning from reconnaissance to exploitation.

D Parameter Derivation and Validation

This appendix documents the derivation and sensitivity analysis for hyperparameters in PENTESTGPT v2.

D.1 Validation Dataset

All hyperparameters are tuned on a held-out validation set of 30 execution traces from retired HTB machines (2022–2023), disjoint from the PentestGPT Benchmark evaluation set. The validation set includes 10 Easy, 12 Medium, and 8 Hard machines, covering web exploitation (12), Linux privilege escalation (10), and Windows/AD attacks (8). We use GPT-4o for validation to avoid overlap with evaluation models (GPT-5.2, Opus 4.5, Gemini 3).

D.2 TDI Weight Selection

Table 13 presents TDI weights derived via grid search over $w \in [0.1, 0.4]$ with step size 0.05, subject to $\sum w_i = 1$. Performance is measured as mean subtask completion rate across the validation set.

Performance varies within $\pm 3\%$ across configurations where all weights remain in $[0.1, 0.4]$, indicating robustness to precise weight selection. The selected configuration ($w_H = w_E = 0.3$, $w_C = w_S = 0.2$) reflects domain intuition: horizon and evidence confidence are primary difficulty signals, while context load and success rate provide secondary modulation.

D.3 Mode Selection Thresholds

Table 14 presents sensitivity analysis for mode selection thresholds (θ_{explore} , θ_{exploit}).

The intermediate zone ($\theta_{\text{exploit}} \leq \text{TDI} \leq \theta_{\text{explore}}$) triggers LLMDECIDE. Narrower zones reduce LLM calls but sacrifice adaptivity; wider zones increase overhead without proportional benefit.

D.4 Pruning Parameters

The pruning threshold ($\theta_{\text{prune}} = 0.8$) and minimum attempts ($k_{\text{min}} = 3$) prevent both premature and excessively delayed pruning.

Lower thresholds increase false pruning (abandoning tractable paths); higher thresholds waste attempts on intractable paths. The selected configuration achieves favorable balance.

D.5 UCB Difficulty Penalty

The difficulty penalty coefficient ($\lambda = 0.5$) modulates how strongly TDI affects node selection in the UCB formula.

$\lambda = 0$ recovers standard UCB, which underperforms due to insufficient difficulty awareness. $\lambda = 1.0$ over-penalizes difficult nodes, preventing exploration of challenging but tractable paths.

D.6 Context Load Degradation Study

To establish the 40% context load threshold, we conduct a controlled study measuring LLM instruction-following accuracy under varying context loads.

Methodology. We construct 50 penetration testing instruction-following tasks from an independent GOAD deployment (separate from evaluation instances). Each task comprises a system state description, accumulated context (tool outputs, discovered information), and a specific instruction (e.g., “Extract the service account password from the Kerberoast output and attempt authentication”). Tasks are designed with unambiguous correct responses, enabling binary accuracy scoring.

For each task, we generate context variants at 10%, 20%, 30%, 40%, 50%, 60%, 70%, 80%, and 90% of the model’s context window. Context padding uses realistic penetration testing artifacts: verbose tool outputs, reconnaissance results, and session histories from actual GOAD runs. Padding is inserted before the instruction to simulate accumulated session context. We evaluate GPT-4o (128K context), Claude-3-Sonnet (200K context), and Gemini-1.5-Pro (1M context) with temperature 0, running each task-context combination three times.

Performance remains stable ($>90\%$) up to 40% load, then degrades approximately linearly. The 40% threshold represents the inflection point beyond which additional context yields diminishing returns and begins actively harming performance.

Failure Mode Analysis. Beyond 40% load, failures concentrate in three categories: ignoring relevant information from earlier context (42% of failures), hallucinating tool outputs not present in context (31%), and executing incorrect but plausible commands (27%). These patterns align with the “lost in the middle” phenomenon documented in prior work [18].

Table 11: Security tools integrated into PENTESTGPT v2. Each tool has a typed interface specifying input schema, output parsing, and execution constraints.

Category	Tool	Description
Reconnaissance	nmap	Network discovery, port/service scanning, OS fingerprinting
	masscan	High-speed port scanner for large networks
	gobuster	Directory/DNS bruteforcing for web discovery
	ffuf	Web fuzzer for directories, parameters, vhosts
	feroxbuster	Recursive web content discovery
	nikto	Web server vulnerability scanner
	whatweb	Web technology fingerprinting
	enum4linux	SMB/Samba enumeration (users, shares, OS)
Web Exploitation	sqlmap	SQL injection detection and exploitation
	burpsuite	Web proxy for traffic interception and testing
	zap	OWASP web vulnerability scanner
	wfuzz	Web fuzzer for parameters and authentication
	commix	Command injection exploitation
	nuclei	Template-based CVE and misconfiguration scanner
Network Exploitation	metasploit	Exploitation framework with pre/post-exploitation modules
	netcat	TCP/UDP networking utility
	crackmapexec	Windows/AD post-exploitation toolkit
	responder	LLMNR/NBT-NS poisoner for credential capture
	evil-winrm	WinRM shell with pass-the-hash support
	chisel	HTTP tunneling for network pivoting
	proxychains	SOCKS/HTTP proxy routing for pivoting
Credential Attacks	hashcat	GPU password cracker (300+ hash types)
	john	Rule-based password cracker
	hydra	Online bruteforcing (50+ protocols)
	impacket	Protocol library (secretsdump, psexec, wmiexec)
	kerbrute	Kerberos user enumeration and password spraying
Active Directory	bloodhound	AD attack path visualization via graph analysis
	sharpbound	BloodHound data collector
	rubeus	Kerberos attack toolkit (roasting, tickets)
	mimikatz	Memory credential extraction
	powerview	AD enumeration PowerShell tool
	ldapdomaindump	LDAP data extraction
	pingcastle	AD security assessment and risk scoring
	adrecon	AD reconnaissance reporting
Privilege Escalation	linpeas	Linux privesc enumeration
	winpeas	Windows privesc enumeration
	pspy	Linux process monitor (cron, scheduled tasks)
	seatbelt	Windows security auditing

Table 12: Evidence confidence scoring rubric. Scores are assigned based on the strongest evidence type at each node; when multiple evidence types are present, the highest applicable score is used.

Evidence Type	Score	Indicators
<i>Verified Evidence (Exploitation Confirmed)</i>		
Valid credentials	1.0	Successful authentication via SSH, WinRM, SMB, or web login
Shell access	1.0	Interactive command execution confirmed
Data exfiltration	1.0	Sensitive data retrieved (flags, database contents, config files)
<i>Confirmed Vulnerability (Exploit Available)</i>		
CVE with public exploit	0.8	Vulnerability scanner confirmation + Exploit-DB/Metasploit module exists
Auth bypass confirmed	0.8	Endpoint accessible without credentials when authentication expected
Injection confirmed	0.8	SQL/command injection produces observable side effects
<i>Plausible Hypothesis (Evidence Supports)</i>		
Version-matched vuln	0.5	Service version matches known vulnerable version range
Configuration weakness	0.5	Misconfiguration identified (default credentials, open permissions)
Information disclosure	0.5	Sensitive information leaked (usernames, paths, internal IPs)
<i>Speculative Hypothesis (Minimal Evidence)</i>		
Service identified	0.3	Port open with service fingerprint, no version/vulnerability match
Potential attack surface	0.3	Endpoint exists but no vulnerability indicators
Unconfirmed assumption	0.3	Hypothesis based on common patterns without direct evidence